# HUFE 数据结构第三讲

**2026 年 3 月 22 日**

## 1 本次课定位

本次课对应 5 次课安排中的第 3 次课，时长 3 小时。本讲在整体安排中原本还要兼顾查找与内排序，但为了让同学先把非线性结构中最综合的一章吃透，这一讲我们先**集中讲图**，把图的概念、存储、遍历、最小生成树和最短路径讲扎实。

图这一章的难点在于：它不像线性表那样只处理前后关系，也不像树那样有天然根结点，而是要同时处理**顶点、边、方向、权值、连通性和算法过程**。因此本讲的核心不是死记定义，而是建立一条稳定分析主线：**概念 - 表示 - 遍历 - 生成树 - 最短路径**。

### 1.1 本次课目标

- 理解图、顶点、边、弧、度、路径、回路、连通图、强连通图等基础概念。
- 区分无向图、有向图、网、完全图、连通分量、强连通分量等常见术语。
- 掌握图的几种存储结构，尤其是邻接矩阵和邻接表，并了解十字链表与邻接多重表。
- 掌握深度优先搜索 DFS 与广度优先搜索 BFS 的思想、过程、复杂度与适用场景。
- 理解生成树、最小生成树的概念，掌握 Prim 和 Kruskal 算法的基本思想。
- 掌握最短路径问题的基本类型，重点理解 Dijkstra 算法的执行过程和使用条件。
- 能处理图这一章中常见的判断题、计算题、算法分析题和结构选择题。

### 1.2 考试导向提醒

图这一章在试卷中通常属于**高频 + 综合性较强**的内容，常见出题方式主要有五类：

- **概念判断题**：如无向图与有向图、连通与强连通、生成树与最小生成树的区别。
- **存储结构题**：如邻接矩阵与邻接表的特点比较、度数统计、稀疏图与稠密图选型。
- **遍历过程题**：给定图和起点，写出 DFS 或 BFS 访问序列。
- **最小生成树题**：要求写出 Prim 或 Kruskal 的选边过程，或求最小生成树总权值。
- **最短路径题**：给定带权图，分析 Dijkstra 每一步的距离更新结果。

所以复习时不能只会背定义，一定要形成“**看结构、会表示、能遍历、能算路径、能比较算法**”的整体能力。

© 2026 isomoes

### 1.3 3 小时建议节奏

| 阶段 | 时间 | 主要内容 |
|---|---|---|
| 第 1 段 | 35 分钟 | 图的基本概念：有向图、无向图、度、路径、连通、生成树等基础术语 |
| 第 2 段 | 40 分钟 | 图的存储结构：邻接矩阵、邻接表、十字链表、邻接多重表及特点比较 |
| 第 3 段 | 45 分钟 | 图的遍历：DFS、BFS 的思想、过程、复杂度与典型访问序列 |
| 第 4 段 | 35 分钟 | 生成树、最小生成树：Prim、Kruskal 的选边规则与适用图形 |
| 第 5 段 | 25 分钟 | 最短路径：Dijkstra 的核心思想、限制条件与典型题 |

## 2 图的基本概念

### 2.1 定义

图是由顶点集 $V$ 和边集 $E$ 组成的二元组，记作：

$$G = (V, E)$$

其中，$V$ 是顶点的有限非空集合，$E$ 是连接顶点之间关系的边的有限集合。

和前面学过的结构相比，图最核心的特点是：**顶点之间的关系是多对多**。也正因为如此，图属于典型的非线性结构。

### 2.2 有向图与无向图

- **无向图**：边没有方向，若顶点 $v_i$ 与 $v_j$ 相连，记作 $(v_i, v_j)$。
- **有向图**：边有方向，此时边也称为**弧**，记作 $\langle v_i, v_j \rangle$，其中 $v_i$ 是弧尾，$v_j$ 是弧头。

考试中要特别注意：

- 在无向图中，边 $(v_i, v_j)$ 和 $(v_j, v_i)$ 表示同一条边。
- 在有向图中，弧 $\langle v_i, v_j \rangle$ 和 $\langle v_j, v_i \rangle$ 是两条不同的弧。

### 2.3 简单图、完全图与网

- **简单图**：不存在重复边，也不存在顶点到自身的边。
- **完全图**：任意两个不同顶点之间都存在边。
- **网**：边或弧上带有权值的图，也叫带权图。

© 2026 isomoes

若无向完全图有 $n$ 个顶点，则边数为：

$$\frac{n(n-1)}{2}$$

若有向完全图有 $n$ 个顶点，则弧数为：

$$n(n-1)$$

这两个公式是非常高频的客观题考点。

## 2.4 顶点的度

- 在**无向图**中，某顶点的**度**是与该顶点相关联的边数。
- 在**有向图**中，某顶点有两个量：**入度**和**出度**。
- 有向图中顶点的度满足：**度 = 入度 + 出度**。

两个常见结论：

1. 在无向图中，所有顶点度数之和等于边数的 2 倍。
2. 在有向图中，所有顶点入度之和等于所有顶点出度之和，且都等于弧数。

## 2.5 路径、回路与简单路径

- **路径**：从一个顶点到另一个顶点经过的顶点序列。
- **路径长度**：路径上边或弧的数目；若是带权图，也可能指路径上权值之和。
- **回路（环）**：起点和终点相同的路径。
- **简单路径**：除起点和终点可能相同外，其余顶点都不重复的路径。
- **简单回路**：除起点终点外，其余顶点不重复的回路。

## 2.6 连通、连通图与强连通图

- 在无向图中，若两个顶点之间存在路径，则称这两个顶点是**连通**的。
- 若无向图中任意两个顶点都连通，则该图称为**连通图**。
- 连通图中的极大连通子图称为**连通分量**。

对有向图：

- 若从顶点 $v_i$ 到 $v_j$ 和从 $v_j$ 到 $v_i$ 都存在路径，则称二者是**强连通**的。
- 若有向图中任意两个顶点都强连通，则称该图为**强连通图**。
- 有向图中的极大强连通子图称为**强连通分量**。

这里最容易混淆的是：**“连通”通常针对无向图，“强连通”通常针对有向图**。

### 2.7 生成树与生成森林

- **生成树**：连通图的一个包含全部顶点的极小连通子图。
- **生成森林**：非连通图各个连通分量的生成树所组成的集合。

若一个连通无向图有 $n$ 个顶点，则它的任意一棵生成树都满足：

- 恰好有 $n-1$ 条边；
- 去掉任意一条边后会变成非连通；
- 再增加任意一条原图中不在树内的边，就会形成回路。

### 2.8 图的基础练习

1. 为什么图属于非线性结构?
2. 无向图和有向图在边的表示上有什么区别?
3. 无向完全图和有向完全图的边数公式分别是什么?
4. 连通图和强连通图分别对应哪类图?
5. 请判断：有向图中只要任意两个顶点之间存在单向路径，就称为强连通图。（　　）

## 3 图的存储结构

### 3.1 为什么存储结构是重点

图这一章中，很多算法都和存储结构直接相关。你选用不同的存储结构，会直接影响：

- 判断两个顶点是否相邻是否方便；
- 找某个顶点全部邻接点是否高效；
- 空间开销大不大；
- 遍历和路径算法的时间复杂度如何计算。

因此图的学习一定不能只会背“有邻接矩阵和邻接表”，而要真正理解它们各自擅长什么。

### 3.2 邻接矩阵

#### 3.2.1 定义

邻接矩阵是用一个二维数组来表示顶点之间邻接关系的存储方式。
若图有 $n$ 个顶点，则定义一个 $n \times n$ 的矩阵 $A$：

- 对无向图，若 $v_i$ 和 $v_j$ 之间有边，则 $A[i][j]=1$ 或对应权值，否则为 0 或 $\infty$。
- 对有向图，若从 $v_i$ 到 $v_j$ 有弧，则 $A[i][j]=1$ 或对应权值，否则为 0 或 $\infty$。

© 2026 isomoes

### 3.2.2 结构定义示意

```c
#define MaxVertexNum 100
typedef struct {
    char Vex[MaxVertexNum];
    int Edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
} MGraph;
```

### 3.2.3 特点

**优点：**

- 判断两个顶点是否相邻非常快，可直接看矩阵元素，时间复杂度为 $O(1)$。
- 容易统计某顶点的度、入度、出度。
- 适合实现稠密图相关算法。

**缺点：**

- 空间复杂度固定为 $O(|V|^2)$，对稀疏图不经济。
- 找某顶点全部邻接点时，通常要扫描整行或整列。

### 3.2.4 高频判断

1. 无向图的邻接矩阵一定是对称矩阵。
2. 无向图中第 $i$ 行元素之和等于顶点 $v_i$ 的度。
3. 有向图中第 $i$ 行元素之和等于顶点 $v_i$ 的出度，第 $i$ 列元素之和等于其入度。

## 3.3 邻接表

### 3.3.1 定义

邻接表是一种顺序存储与链式存储相结合的结构。它为每个顶点建立一个表头，再用链表记录该顶点的所有邻接点。

### 3.3.2 结构定义示意

```c
typedef struct ArcNode {
    int adjvex;
    struct ArcNode *next;
} ArcNode;

typedef struct VNode {
```

```
    VertexType data;
    ArcNode *first;
} VNode, AdjList[MaxVertexNum];

typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;
```

### 3.3.3 特点

**优点：**

- 空间复杂度约为 $O(|V|+|E|)$，更适合稀疏图。
- 找某顶点的全部邻接点方便。
- 在 DFS、BFS 中常更高效。

**缺点：**

- 判断两个顶点是否直接相邻，不如邻接矩阵直观。
- 若要查找某一条边是否存在，通常需要遍历对应链表。

### 3.3.4 无向图与有向图中的理解差异

- 在无向图的邻接表中，一条边通常会在两个顶点的链表中各出现一次。
- 在有向图的邻接表中，一条弧通常只会出现在弧尾对应的链表中一次。

因此：

- 无向图中所有链表结点总数通常为 $2|E|$。
- 有向图中所有链表结点总数通常为 $|E|$。

## 3.4 十字链表

### 3.4.1 为什么会有十字链表

对于有向图，如果既希望方便找某顶点的**出边**，又希望方便找某顶点的**入边**，普通邻接表就不够直接了。

这时常用**十字链表**来存储有向图。

### 3.4.2 核心思想

十字链表中，每条弧结点同时挂到两条链上：

- 一条是同一弧尾的出边链；
- 一条是同一弧头的入边链。

这样就可以同时方便地查找某顶点的出边和入边。

### 3.4.3 适用对象

- 十字链表主要用于**有向图**。
- 它是“了解 + 识记”型考点，常考“适合存储什么图”“能方便求什么量”。

## 3.5 邻接多重表

### 3.5.1 为什么会有邻接多重表

在无向图中，一条边会分别出现在两个顶点的邻接表中，因此删除某条边时会涉及两个链表位置，不够自然。

为了解决这个问题，可以使用**邻接多重表**。

### 3.5.2 核心思想

邻接多重表让**每条边只对应一个边结点**，但这个边结点同时链接到与该边相关的两个顶点边表中。

它的好处是：

- 对无向图中的边进行共享表示；
- 删除边、标记边等操作更方便。

### 3.5.3 适用对象

- 邻接多重表主要用于**无向图**。
- 这也是常见的识记型考点。

## 3.6 几种存储结构对比

| 存储结构 | 适合图类型 | 主要优点 | 主要缺点 |
|---|---|---|---|
| 邻接矩阵 | 稠密图 | 判断相邻快，结构直观 | 空间开销大 |
| 邻接表 | 稀疏图 | 节省空间，找邻接点方便 | 判断相邻不如矩阵快 |
| 十字链表 | 有向图 | 同时便于找入边和出边 | 结构较复杂 |
| 邻接多重表 | 无向图 | 一条边只存一次 | 结构较复杂 |

© 2026 isomoes

### 3.7 存储结构练习

1. 邻接矩阵为什么更适合稠密图?
2. 邻接表为什么更适合稀疏图?
3. 十字链表主要用于有向图还是无向图?
4. 邻接多重表主要用于有向图还是无向图?
5. 请判断：无向图的邻接矩阵一定是对称矩阵。(　　)

# 4 图的遍历

## 4.1 为什么图的遍历比树更难

树在遍历时天然没有回路，且通常有根结点；图则不同：

- 图中可能存在回路；
- 图不一定连通；
- 同一个顶点可能通过多条路径被再次访问。

因此图的遍历必须解决一个关键问题：**防止重复访问**。这就是为什么 DFS 和 BFS 都需要设置访问标记数组 `visited`。

## 4.2 深度优先搜索 DFS

### 4.2.1 思想

深度优先搜索的基本思想是：从某个起始顶点出发，沿着一条路径尽可能往深处走，直到不能再继续，然后回退到上一个顶点，改走其他分支。

它的直观感觉类似于“走迷宫先一条路走到底，不通再回头”。

### 4.2.2 递归写法示意

```
void DFS(Graph G, int v) {
    visit(v);
    visited[v] = 1;
    for (w = FirstNeighbor(G, v); w >= 0; w = NextNeighbor(G, v, w)) {
        if (!visited[w]) {
            DFS(G, w);
        }
    }
}
```

### 4.2.3 特点

- 常用递归实现，也可用栈实现。
- 在形式上与树的先序遍历比较接近。
- 适合分析连通性、寻找路径、构造 DFS 生成树等问题。

### 4.2.4 复杂度

- 采用邻接矩阵时，时间复杂度通常为 $O(|V|^2)$。
- 采用邻接表时，时间复杂度通常为 $O(|V|+|E|)$。

## 4.3 广度优先搜索 BFS

### 4.3.1 思想

广度优先搜索的基本思想是：从起始顶点出发，先访问距离它最近的一层顶点，再访问更远的一层顶点，按“由近到远”逐层扩展。

它和树的层序遍历非常接近，因此通常借助**队列**实现。

### 4.3.2 队列写法示意

```
void BFS(Graph G, int v) {
    visit(v);
    visited[v] = 1;
    EnQueue(Q, v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q, u);
        for (w = FirstNeighbor(G, u); w >= 0; w = NextNeighbor(G, u, w)) {
            if (!visited[w]) {
                visit(w);
                visited[w] = 1;
                EnQueue(Q, w);
            }
        }
    }
}
```

### 4.3.3 特点

- 常借助队列实现。
- 与树的层序遍历联系最紧密。
- 在**无权图单源最短路径**问题中非常重要。

#### 4.3.4 复杂度

- 采用邻接矩阵时，时间复杂度通常为 $O(|V|^2)$。
- 采用邻接表时，时间复杂度通常为 $O(|V|+|E|)$。

### 4.4 DFS 与 BFS 对比

| 比较项目 | DFS | BFS |
|---|---|---|
| 核心思想 | 一条路走到底再回溯 | 一层一层向外扩展 |
| 常用辅助结构 | 栈或递归 | 队列 |
| 形式类比 | 树的先序遍历 | 树的层序遍历 |
| 典型用途 | 连通性、路径搜索 | 分层访问、无权最短路 |

### 4.5 非连通图怎么遍历

如果图不是连通图，只从一个顶点出发做一次 DFS 或 BFS，通常不能访问全部顶点。因此完整遍历图时，通常要写成：

1. 先把 `visited` 全部初始化为未访问；
2. 依次检查每个顶点；
3. 若该顶点尚未访问，就以它为起点再做一次 DFS 或 BFS。

这一点很容易在选择题里出错。

### 4.6 遍历练习

1. 图的遍历为什么必须设置访问标记?
2. DFS 和 BFS 分别常借助什么辅助结构?
3. 为什么说 BFS 和树的层序遍历很接近?
4. 在邻接表上实现遍历时，时间复杂度通常是多少?
5. 请判断：对任意图，从任一顶点出发做一次 BFS 一定能访问全部顶点。（　　）

## 5 生成树与最小生成树

### 5.1 生成树再理解

在连通无向图中，生成树的重点不是“任选一些边”，而是同时满足两个条件：

- 覆盖全部顶点；
- 保持连通且不形成回路。

如果原图有 $n$ 个顶点，那么生成树必须有 $n-1$ 条边，这一点一定要记牢。

## 5.2 最小生成树的定义

最小生成树（MST）是连通加权无向图中，所有生成树里**边权值之和最小**的一棵。
这里有两个关键词：

- 必须是**连通加权无向图**；
- 比较标准是**总权值最小**，不是边数最少，因为所有生成树边数本来都相同。

## 5.3 Prim 算法

### 5.3.1 思想

Prim 算法的思路可以概括为：

1. 从某个顶点出发，先把它看成当前生成树的一部分；
2. 每一步都从“已选顶点集合”和“未选顶点集合”之间，选一条权值最小的边；
3. 把新顶点并入生成树；
4. 重复直到所有顶点都加入。

它的特点是：**逐点扩张，像一棵树慢慢长大**。

### 5.3.2 过程示意

```
void Prim(MGraph G, int start) {
    初始化已选顶点集合U = {start};
    while (U 中顶点数 < G.vexnum) {
        在一个端点属于U、另一个端点不属于U的所有边中
        选权值最小的一条;
        把新顶点和该边加入生成树;
    }
}
```

### 5.3.3 适用场景

- Prim 更适合**稠密图**。
- 若采用邻接矩阵，写法较自然。

## 5.4 Kruskal 算法

### 5.4.1 思想

Kruskal 算法的思路可以概括为：

1. 先把所有边按权值从小到大排序；
2. 从最小边开始依次考察；
3. 若当前边加入后不会形成回路，就把它选入；
4. 直到选出 $n-1$ 条边为止。

它的特点是：**逐边筛选，能选就选，但绝不成环**。

### 5.4.2 过程示意

```
void Kruskal(Graph G) {
    将所有边按权值从小到大排序;
    count = 0;
    for (每条边e按序考察) {
        if (加入e后不形成回路) {
            选入e;
            count++;
        }
        if (count == G.vexnum - 1)
            break;
    }
}
```

### 5.4.3 适用场景

- Kruskal 更适合**稀疏图**。
- 它常与并查集思想联系起来理解，但考试中多数情况只要求掌握选边过程。

## 5.5 Prim 与 Kruskal 对比

| 比较项目 | Prim | Kruskal |
|---|---|---|
| 选边视角 | 从顶点集合向外扩张 | 从全局边集中依次选边 |
| 核心限制 | 新边必须连接已选点和未选点 | 新边不能形成回路 |
| 更适合 | 稠密图 | 稀疏图 |
| 思维特征 | 长树 | 选边 |

## 5.6 最小生成树常见判断

1. 最小生成树一定是生成树，因此一定包含全部顶点。
2. 最小生成树只针对连通加权无向图讨论。
3. 最小生成树不一定唯一，但最小权值和相同。
4. 生成树边数固定为 $n-1$，不是越少越好，而是在固定边数条件下总权值最小。

© 2026 isomoes

### 5.7 最小生成树练习

1. 生成树为什么一定没有回路?
2. 连通无向图有 $n$ 个顶点时，其生成树有多少条边?
3. Prim 和 Kruskal 的核心选边规则各是什么?
4. 哪个算法更适合稀疏图?
5. 请判断：最小生成树一定唯一。（　　）

## 6 最短路径

### 6.1 最短路径问题的基本理解

最短路径研究的是：从一个顶点到另一个顶点，怎样走总代价最小。
这里的“最短”可能有两种理解：

- 在无权图中，通常指经过的边数最少；
- 在带权图中，通常指路径上边权值之和最小。

### 6.2 BFS 与无权最短路径

若图是无权图，或者所有边权都可视为相同，那么从某起点到其他顶点的最短路径问题，通常可由 **BFS** 解决。
原因是 BFS 按层次扩展，先到达的顶点一定是通过更少边数到达的。
这也是为什么很多选择题会把“BFS”和“无权图最短路径”放在一起考。

### 6.3 Dijkstra 算法

#### 6.3.1 适用条件

Dijkstra 算法用于求解**单源最短路径**问题，即从一个源点到其他各顶点的最短路径。
但它有一个非常重要的限制：

**Dijkstra 只适用于边权非负的图。**

若图中存在负权边，则不能直接使用 Dijkstra。

#### 6.3.2 核心思想

它的基本思想可以概括为：

1. 先确定源点到自身距离为 0，到其他点距离为初值；
2. 每次从“尚未确定最短路径的顶点”中，选出当前距离最小的一个顶点；

3. 把该顶点的最短距离正式确定下来；
4. 再利用这个顶点去更新其邻接点的距离估计值；
5. 重复直到所有顶点处理完毕。

其中最关键的一句话是：**每一步都选当前暂时距离最小且未确定的顶点**。

### 6.3.3 算法过程示意

```
void Dijkstra(MGraph G, int v0) {
    初始化dist数组和visited数组;
    dist[v0] = 0;
    for (i = 0; i < G.vexnum; i++) {
        选择当前未确定且dist最小的顶点u;
        visited[u] = 1;
        for (u的每个邻接点v) {
            if (!visited[v] && dist[u] + weight(u, v) < dist[v]) {
                dist[v] = dist[u] + weight(u, v);
            }
        }
    }
}
```

### 6.3.4 理解要点

- `dist[i]` 表示当前已知的源点到顶点 $i$ 的最短距离估计值。
- `visited[i]` 表示该顶点的最短距离是否已经最终确定。
- 一旦某顶点被选为当前最小距离顶点并确定下来，在非负权前提下，它的最短路径长度就不会再变。

## 6.4 Floyd 算法简单了解

如果题目问的是**任意两点之间**的最短路径，则更常见的算法是 Floyd 算法。

它属于多源最短路径算法，时间复杂度为 $O(|V|^3)$。在 HUFE 这一轮复习中，可以先把重点放在 **Dijkstra 的思想和过程**上，而 Floyd 只作概念了解即可。

## 6.5 最短路径练习

1. 无权图的单源最短路径通常可以用什么方法求?
2. Dijkstra 解决的是单源最短路径还是多源最短路径?
3. Dijkstra 为什么不能直接处理负权边?
4. 若题目要求任意两点之间的最短路径，通常会想到哪种算法?

5. 请判断：Dijkstra 算法适用于任意带权图。（　　）

# 7 易混点总结

1. **无向图与有向图**：一个边无方向，一个弧有方向，表示方法不能混。
2. **连通与强连通**：前者主要针对无向图，后者主要针对有向图。
3. **邻接矩阵与邻接表**：前者判断相邻快，后者找邻接点更省空间。
4. **DFS 与 BFS**：DFS 像走迷宫走到底，BFS 像一层一层扩散。
5. **生成树与最小生成树**：最小生成树首先必须是生成树，然后才比较权值和。
6. **Prim 与 Kruskal**：一个从点集出发扩张，一个从边集出发筛选。
7. **BFS 与 Dijkstra**：前者常处理无权最短路，后者处理非负权单源最短路。
8. **十字链表与邻接多重表**：十字链表偏有向图，邻接多重表偏无向图。

# 8 本讲知识框架

建议把本讲内容整理成下面这条主线：

| 主题 | 核心问题 | 典型内容 | 高频考点 |
|---|---|---|---|
| 图基础 | 顶点如何建立关系 | 有向图、无向图、度、路径、连通 | 术语辨析、边数计算 |
| 存储结构 | 图在内存中怎样表示 | 邻接矩阵、邻接表 | 稠密/稀疏图选型、度统计 |
| 图遍历 | 如何不重不漏访问图 | DFS、BFS | 访问序列、复杂度、辅助结构 |
| 生成树 | 如何保持连通又无环 | 生成树、MST | 边数、Prim、Kruskal |
| 最短路径 | 怎样走代价最小 | BFS、Dijkstra、Floyd | 无权最短路、负权限制 |

# 9 典型例题

## 9.1 例题 1：无向完全图边数

**题目：**一个无向完全图有 6 个顶点，则共有多少条边？
**分析：**无向完全图边数公式为 $\frac{n(n-1)}{2}$。
**解：**

$$\frac{6\times 5}{2}=15$$

**答案：**15 条边。

© 2026 isomoes

### 9.2 例题 2：邻接矩阵中的度

**题目：** 某无向图的邻接矩阵中，第 3 行元素之和为 4，则顶点 $v_3$ 的度是多少?
**分析：** 无向图邻接矩阵中某一行元素之和就是该顶点的度。
**答案：** 4。

### 9.3 例题 3：遍历辅助结构

**题目：** 图的广度优先搜索通常借助什么线性结构实现?
**分析：** 广度优先要求按先到先处理的顺序逐层扩展，符合先进先出。
**答案：** 队列。

### 9.4 例题 4：生成树边数

**题目：** 一个连通无向图有 8 个顶点，则其任意生成树有多少条边?
**分析：** 连通无向图的生成树边数恒为 $n-1$。
**答案：** 7 条边。

### 9.5 例题 5：Prim 与 Kruskal 的区别

**题目：** Prim 和 Kruskal 都用于求最小生成树，它们的主要思路有什么不同?
**分析：** Prim 从顶点集合向外扩张，Kruskal 从全局边集按权值筛选且避免成环。
**答案：** Prim 是“逐点扩张”，Kruskal 是“逐边选择”。

### 9.6 例题 6：Dijkstra 的使用条件

**题目：** 某带权图中存在负权边，是否还能直接使用 Dijkstra 算法?
**分析：** Dijkstra 建立在“已确定的最短距离不再变化”的非负权前提上。
**答案：** 不能。

### 9.7 例题 7：BFS 解决什么最短路

**题目：** 无权图中，从某一源点到其余顶点的最短路径问题，通常优先使用 DFS 还是 BFS?
**分析：** 无权图最短路径等价于边数最少，而 BFS 按层次扩展。
**答案：** BFS。

## 10 课堂练习

### 10.1 基础练习

1. 图和树相比，为什么说图的关系更复杂?
2. 有向图中顶点的度、入度、出度三者有什么关系?
3. 为什么无向图邻接矩阵一定是对称矩阵?

© 2026 isomoes

4. 邻接表为什么通常更适合稀疏图?

5. DFS 和 BFS 在访问策略上的根本差异是什么?

## 10.2 判断与选择

1. 图中任意两个顶点之间都存在边，则该图一定是完全图。（　　）

2. 无向图中所有顶点度数之和等于边数。（　　）

3. 有向图中所有顶点入度之和等于所有顶点出度之和。（　　）

4. BFS 通常借助栈实现。（　　）

5. 最小生成树一定包含原图全部顶点。（　　）

6. Dijkstra 可以直接处理负权边。（　　）

## 10.3 计算与分析

1. 一个无向完全图有 5 个顶点，求边数。

2. 一个有向完全图有 4 个顶点，求弧数。

3. 一个连通无向图有 10 个顶点，求任意生成树的边数。

4. 为什么在图遍历中必须设置 `visited` 数组?

5. Prim 算法和 Kruskal 算法都求最小生成树，但各自的选边标准有什么不同?

6. 为什么说 BFS 常可以求无权图的单源最短路径?

## 10.4 综合小题

**题目：**某校园系统要处理三个任务：

- 任务 A：表示校园楼宇之间的道路连接关系，并经常判断两栋楼是否直接相连；
- 任务 B：从某一教学楼开始，按“距离越来越远”的层次依次检查周边楼宇；
- 任务 C：在所有楼宇都连通的前提下，选择一组总造价最低的道路方案。

请分别指出任务 A、任务 B、任务 C 对应的核心数据结构或算法思想，并说明理由。

© 2026 isomoes

# 11 课后小测参考答案

1. 因为图中一个顶点可以和多个顶点发生关系，整体是多对多结构。
2. 有向图中顶点的度 = 入度 + 出度。
3. 因为无向图中边 $(v_i, v_j)$ 与 $(v_j, v_i)$ 表示同一条边，所以矩阵关于主对角线对称。
4. 因为空间复杂度更接近 $O(|V| + |E|)$，对稀疏图更经济。
5. DFS 强调先深入再回溯，BFS 强调按层次逐步扩展。
6. 第 1 题对；第 2 题错；第 3 题对；第 4 题错；第 5 题对；第 6 题错。
7. 无向完全图边数为 $\frac{5\times4}{2} = 10$。
8. 有向完全图弧数为 $4 \times 3 = 12$。
9. 生成树边数为 $10 - 1 = 9$。
10. 因为图中可能存在回路和重复到达路径，若不标记已访问顶点就可能重复访问甚至死循环。
11. Prim 每次选连接已选点集和未选点集的最小边；Kruskal 每次从全局最小边开始选且不允许成环。
12. 因为 BFS 按边数从少到多逐层扩展，先访问到的顶点对应的路径边数最少。

综合小题参考：任务 A 若经常判断两栋楼是否直接相连，可优先考虑邻接矩阵；任务 B 对应 BFS，因为是按层次向外扩展；任务 C 对应最小生成树思想，可用 Prim 或 Kruskal 算法。

## 11.1 分节练习参考答案

**图的基础练习参考：**

1. 因为顶点之间是多对多关系，不是一对一或一对多关系。
2. 无向图用无序对表示边，有向图用有序对表示弧。
3. 无向完全图为 $\frac{n(n-1)}{2}$，有向完全图为 $n(n-1)$。
4. 连通图对应无向图，强连通图对应有向图。
5. 错。强连通要求任意两点互相可达，而不是只存在单向路径。

**存储结构练习参考：**

1. 因为邻接矩阵空间固定为 $O(|V|^2)$，稠密图中空间浪费相对较少，且判断相邻很快。
2. 因为邻接表空间更接近 $O(|V| + |E|)$，当边少时更节省空间。
3. 十字链表主要用于有向图。

4. 邻接多重表主要用于无向图。

5. 对。

**遍历练习参考：**

1. 因为图中可能有回路，也可能通过不同路径多次到达同一顶点。

2. DFS 常借助栈或递归，BFS 常借助队列。

3. 因为它们都按距离起点越来越远的层次顺序访问结点或顶点。

4. 在邻接表上通常为 $O(|V|+|E|)$。

5. 错。若图不连通，一次 BFS 不能访问全部顶点。

**最小生成树练习参考：**

1. 因为生成树要求在保持连通的同时不形成回路。

2. $n-1$ 条边。

3. Prim 是从已选点集向外选最小边，Kruskal 是按权值从小到大选边且避免成环。

4. Kruskal 更适合稀疏图。

5. 错。若最小权值组合不唯一，则最小生成树也可能不唯一。

**最短路径练习参考：**

1. BFS。

2. 单源最短路径。

3. 因为负权边会破坏“当前最小距离一经确定就不再变化”的基础。

4. Floyd 算法。

5. 错。Dijkstra 只适用于边权非负的图。

## 12 本讲小结

本讲真正要建立的不是一堆零散名词，而是一种看图问题的稳定方法：**先看顶点和边的关系，再看用什么结构存，再看如何遍历，最后再上升到生成树和最短路径问题**。

如果说前两讲分别在建立线性结构和树形结构的理解，那么图这一讲就是把“多对多关系 + 算法过程”真正引进来。这里最需要反复练习的四个点分别是：**邻接矩阵与邻接表的区别、DFS 和 BFS 的访问思想、Prim 与 Kruskal 的选边逻辑、Dijkstra 的使用条件与更新过程**。

下一步如果继续补 class3 后半部分内容，就可以在此基础上接上**查找和内排序**，形成第 3 次课的完整数据结构收尾讲义。